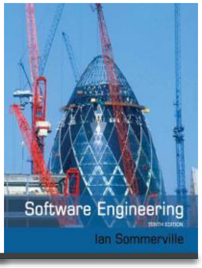
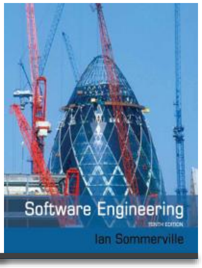


Chapter 1- Introduction

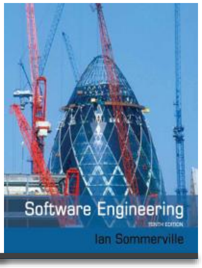
Topics covered



✧ Professional software development

✧ Software engineering ethics

✧ Case studies



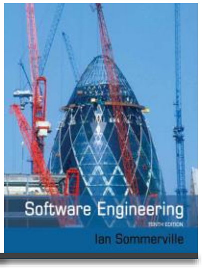
Software engineering

- ✧ The economies of ALL developed nations are dependent on software.
- ✧ More and more systems are software-controlled
- ✧ **Software engineering** is concerned with theories, methods, and tools for professional software development.

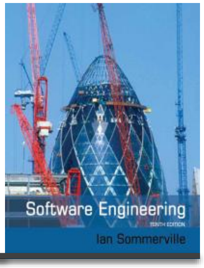
Software costs

- ✧ Software costs often dominate computer system costs.
- ✧ Software costs more to maintain than it does to develop.
- ✧ Software engineering is concerned with cost-effective software development.

Software project failure



✧ *What are the consequences of not using software engineering methods?*



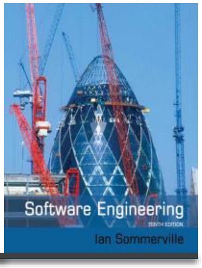
Professional software development

Software products

✧ Generic products

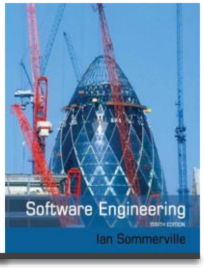
✧ Customized products

Essential attributes of good software



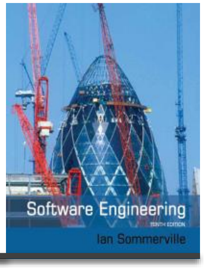
✧ What are the essential attributes of good software?

Essential attributes of good software



Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software engineering



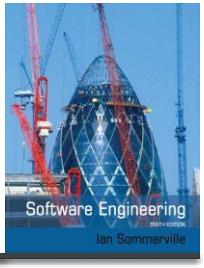
- **Software engineering** is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.

- Engineering discipline
  - Using appropriate **theories** and **methods** to solve problems, bearing in mind **organizational** and **financial constraints**.

Software process activities

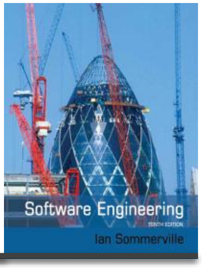
✧ What are the software process activities?

Software process activities



- Software specification
- Software development
- Software validation
- Software evolution

General issues that affect software



- What are the general issues that affect software?

General issues that affect software

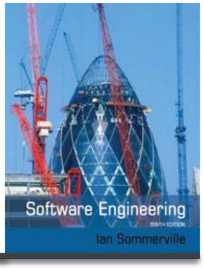
- ✧ Heterogeneity
- ✧ Business and social change
- ✧ Security and trust
- ✧ Scale

Application types

- What are the types of applications?

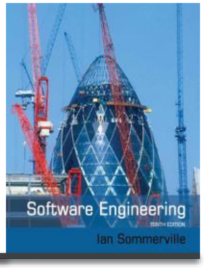
Application types

- ✧ Stand-alone applications
- ✧ Interactive transaction-based applications
- ✧ Embedded control systems
- ✧ Batch processing systems
- ✧ Entertainment systems
- ✧ Systems for modeling and simulation
- ✧ Data collection systems
- ✧ Systems of systems



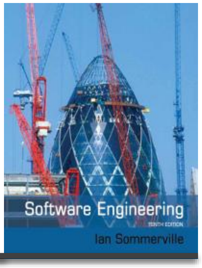
Software engineering ethics

Software engineering ethics & responsibilities



- Software engineering involves wider **responsibilities** than simply the application of technical skills.
- Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law but involves following **a set of principles that are morally correct.**

Issues of professional responsibility

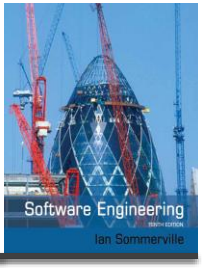


✧ What are the issues of professional responsibility

Issues of professional responsibility

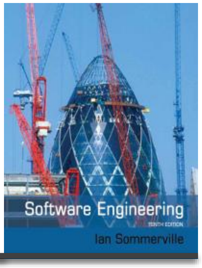
- Confidentiality
- Competence
- Intellectual property rights
- Computer misuse

ACM/IEEE Code of Ethics



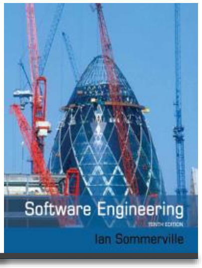
- The professional societies in the US have cooperated to produce a code of ethical practice.
- Members of these organisations sign up to the code of practice when they join.
- The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Rationale for the code of ethics



- *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

The ACM/IEEE Code of Ethics



Software Engineering Code of Ethics and Professional Practice

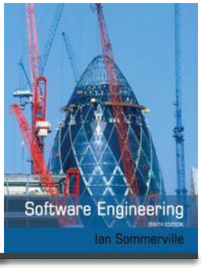
ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Ethical principles



1. **PUBLIC** - Software engineers shall act consistently with the public interest.

2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the best interests of their client and employer, consistent with the public interest.

3. **PRODUCT** - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

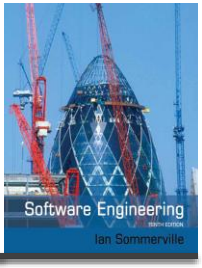
4. **JUDGMENT** - Software engineers shall maintain integrity and independence in their professional judgment.

5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

6. **PROFESSION** - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

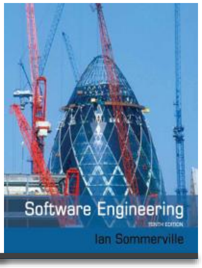
7. **COLLEAGUES** - Software engineers shall be fair to and supportive of their colleagues.

8. **SELF** - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.



Case studies

Ethical dilemmas



- Disagreement in principle with the policies of senior management.
- Your employer acts in an unethical way and releases a safety-critical system without finishing the testing of the system.
- Participation in the development of military weapons systems or nuclear systems.

Case studies

✧ **A personal insulin pump**

- An embedded system in an insulin pump used by diabetics to maintain blood glucose control.

✧ **A mental health case patient management system**

- Mentcare. A system used to maintain records of people receiving care for mental health problems.

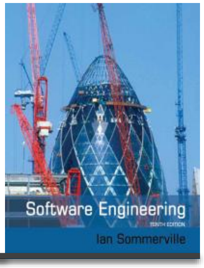
✧ **A wilderness weather station**

- A data collection system that collects data about weather conditions in remote areas.

✧ **iLearn: a digital learning environment**

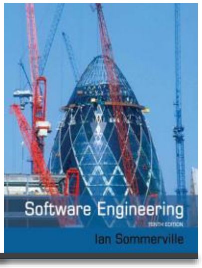
- A system to support learning in schools

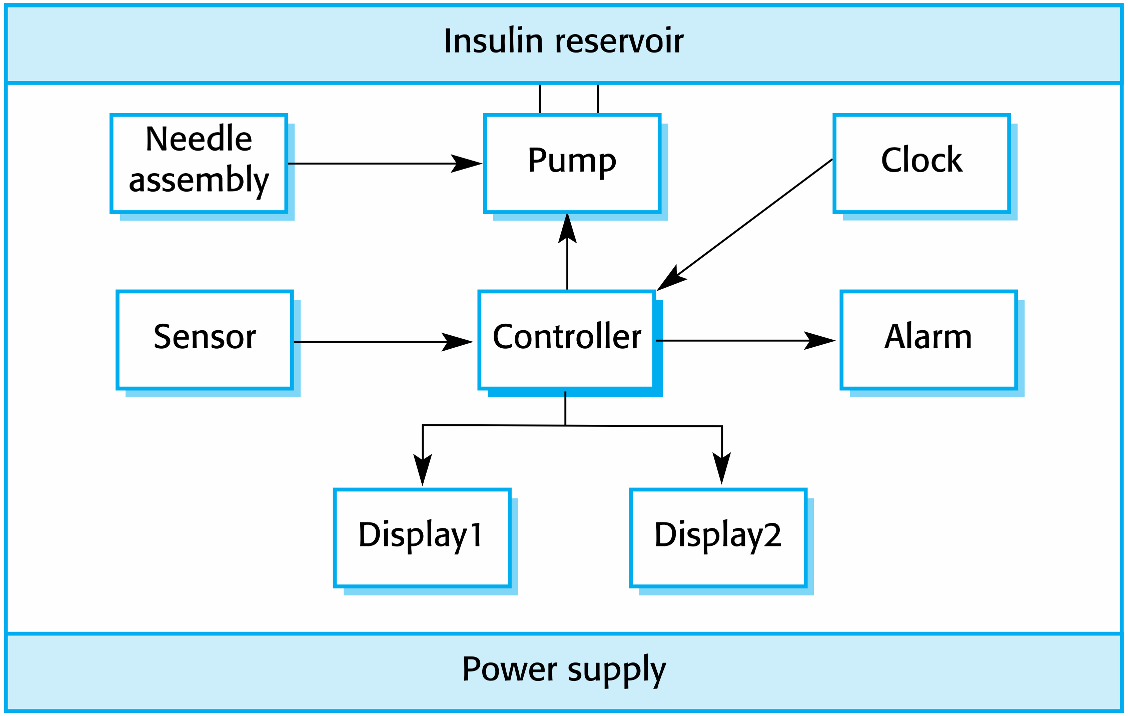
Insulin pump control system



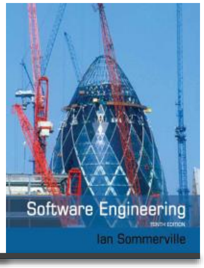
- Collects data from a blood sugar sensor and calculates the amount of insulin required to be injected.
- Calculation based on the rate of change of blood sugar levels.
- Sends signals to a micro-pump to deliver the correct dose of insulin.
- Safety-critical system:
  - **Low blood sugars can lead to brain malfunctioning, coma, and death**;
  - **High blood sugar levels have long-term consequences such as eye and kidney damage**.

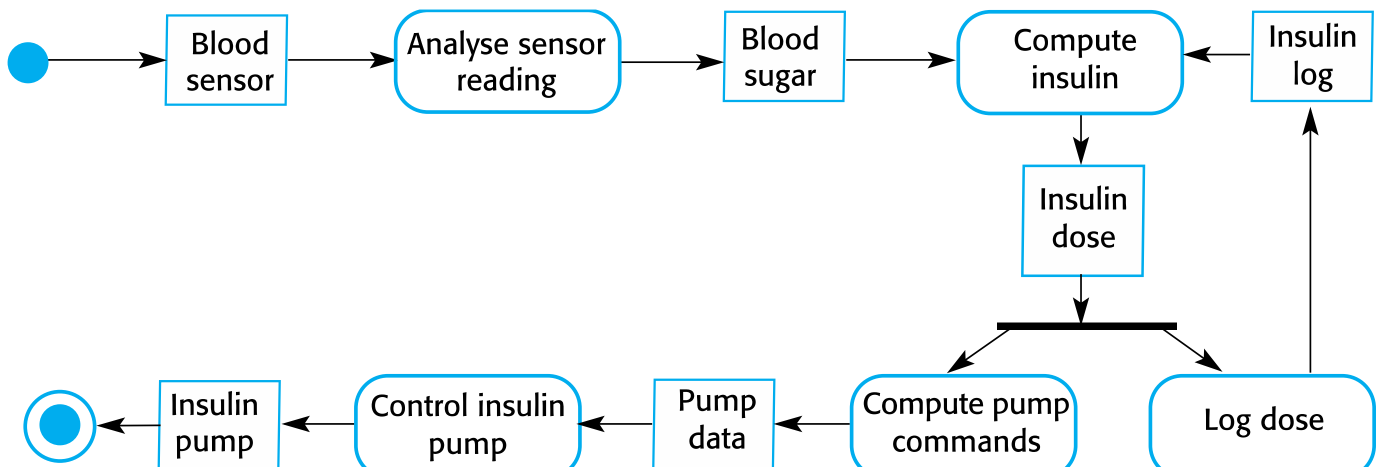
Insulin pump hardware architecture



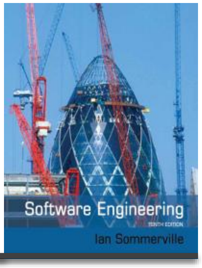


Activity model of the insulin pump



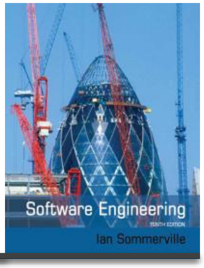


Insulin pump: Essential high-level requirements & concerns



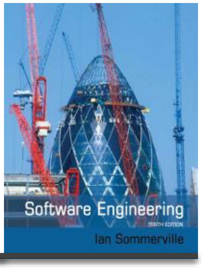
- The system shall be **available** to deliver insulin when required.
- The system shall perform **reliably** and deliver the **correct amount of insulin** to counteract the current level of blood sugar.
- The system must therefore be designed and implemented to ensure that it always **meets these requirements.**

Mentcare: A patient information system for mental health care



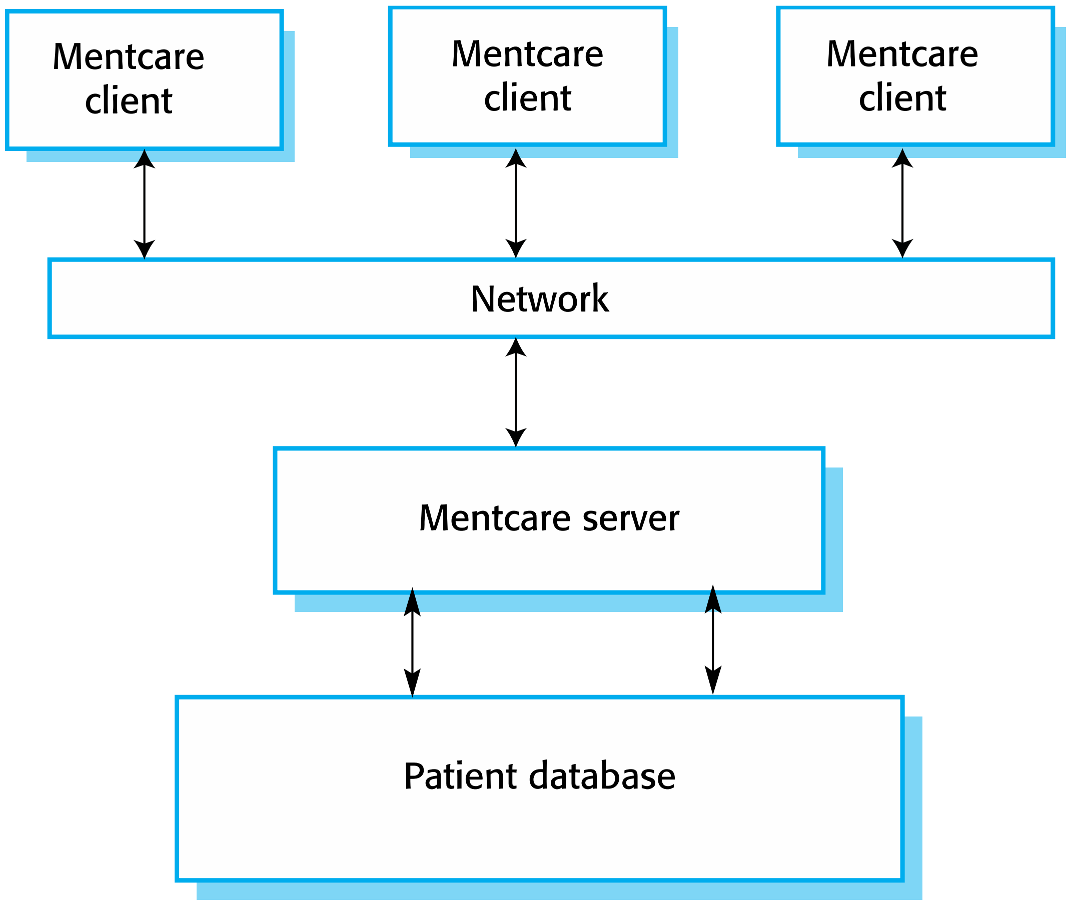
- A patient information system to support mental health care is a medical information system that maintains information about patients suffering from mental health problems and the treatments that they have received.
- Most mental health patients do not require dedicated hospital treatment but need to attend specialist clinics regularly where they can meet a doctor who has detailed knowledge of their problems.
- To make it easier for patients to attend, these clinics are not just run in hospitals. They may also be held in local medical practices or community centres.

Mentcare



- ✧ Mentcare is an information system that is intended for use in clinics.
- ✧ It makes use of a centralized database of patient information but has also been designed to run on a PC, so that it may be accessed and used from sites that do not have secure network connectivity.
- ✧ When the local systems have secure network access, they use patient information in the database, but they can download and use local copies of patient records when they are disconnected.

The organization of the Mentcare system



Key features of the Mentcare system

- **Individual care management**
  - Clinicians can create records for patients, edit the information in the system, view patient history, etc. The system supports data summaries so that doctors can quickly learn about the key problems and treatments that have been prescribed.
- **Patient monitoring**
  - The system monitors the records of patients who are involved in treatment and issues warnings if possible problems are detected.
- **Administrative reporting**
  - The system generates monthly management reports showing the number of patients treated at each clinic, the number of patients who have entered and left the care system, the number of patients sectioned, the drugs prescribed and their costs, etc.

Mentcare: Essential high-level requirements & concerns

✧ Privacy

- It is essential that patient information is **confidential** and is never disclosed to anyone apart from authorised medical staff and the patient themselves.

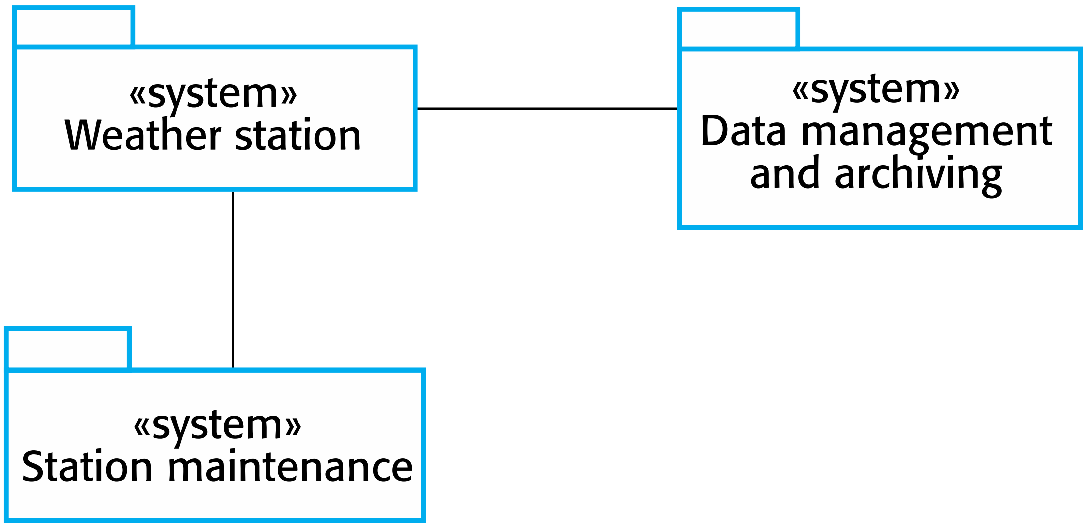
✧ Safety

- Some mental illnesses cause patients to become suicidal or a danger to other people. Wherever possible, the system should **warn medical staff about potentially suicidal or dangerous patients**.
- The system must be **available** when needed otherwise safety may be compromised and it may be impossible to prescribe the correct medication to patients.

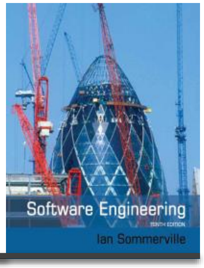
Wilderness weather station

- ✧ The government of a country with large areas of wilderness decides to deploy several hundred weather stations in remote areas.
- ✧ Weather stations collect data from a set of instruments that measure temperature and pressure, sunshine, rainfall, wind speed and wind direction.
  - The weather station includes a number of instruments that measure weather parameters such as the wind speed and direction, the ground and air temperatures, the barometric pressure and the rainfall over a 24-hour period. Each of these instruments is controlled by a software system that takes parameter readings periodically and manages the data collected from the instruments.

The weather station's environment



Weather information system



- ✧ The weather station system
  - This is responsible for collecting weather data, carrying out some initial data processing and transmitting it to the data management system.
- ✧ The data management and archiving system
  - This system collects the data from all of the wilderness weather stations, carries out data processing and analysis and archives the data.
- ✧ The station maintenance system
  - This system can communicate by satellite with all wilderness weather stations to monitor the health of these systems and provide reports of problems.

Additional software functionality

✧ Monitor the instruments, power and communication hardware and report faults to the management system.

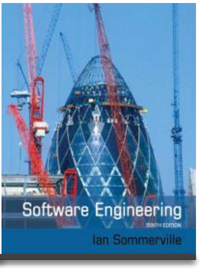
✧ Manage the system power, ensuring that batteries are charged whenever the environmental conditions permit but also that generators are shut down in potentially damaging weather conditions, such as high wind.

✧ Support dynamic reconfiguration where parts of the software are replaced with new versions and where backup instruments are switched into the system in the event of system failure.

iLearn: A digital learning environment

- A digital learning environment is a framework in which a set of general-purpose and specially designed tools for learning may be embedded plus a set of applications that are geared to the needs of the learners using the system.
- The tools included in each version of the environment are chosen by teachers and learners to suit their specific needs.
  - These can be general applications such as spreadsheets, learning management applications such as a Virtual Learning Environment (VLE) to manage homework submission and assessment, games and simulations.

Service-oriented systems

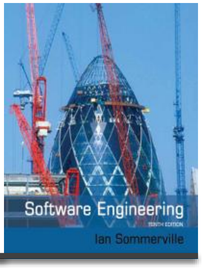


- The system is a service-oriented system with all system components considered to be a replaceable service.
- This allows the system to be updated incrementally as new services become available.
- It also makes it possible to rapidly configure the system to create versions of the environment for different groups such as very young children who cannot read, senior students, etc.

iLearn services

- *Utility services* that provide basic application-independent functionality and which may be used by other services in the system.
- *Application services* that provide specific applications such as email, conferencing, photo sharing etc. and access to specific educational content such as scientific films or historical resources.
- *Configuration services* that are used to adapt the environment with a specific set of application services and do define how services are shared between students, teachers and their parents.

iLearn architecture



Browser-based user interface | iLearn app

Configuration services

Group management | Application management | Identity management

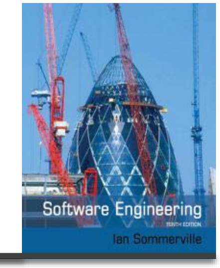
Application services

Email | Messaging | Video conferencing | Newspaper archive
Word processing | Simulation | Video storage | Resource finder
Spreadsheet | Virtual learning environment | History archive

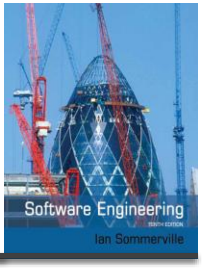
Utility services

Authentication | Logging and monitoring | Interfacing
User storage | Application storage | Search

iLearn service integration



- *Integrated services* are services which offer an API (application programming interface) and which can be accessed by other services through that API. Direct service-to-service communication is therefore possible.
- *Independent services* are services which are simply accessed through a browser interface and which operate independently of other services. Information can only be shared with other services through explicit user actions such as copy and paste; re-authentication may be required for each independent service.



LTD

- **01**
- Using the three case studies – Insulin Pump, Mentcare, and Wilderness Weather Station – evaluate the potential ethical issues that could arise in each system. Which principles of the ACM/IEEE Code of Ethics are most relevant to these issues, and why?
- **02**
- Propose a realistic ethical dilemma that a software engineer might encounter while developing the iLearn digital learning environment. Explain how the engineer should apply the ACM/IEEE Code of Ethics to resolve the dilemma, and justify the chosen course of action.

LTD - 1

- **03**
- Analyze the three ethical dilemmas presented in Slide 26: (1) disagreement in principle with senior management policies, (2) releasing an untested safety-critical system, and (3) participation in developing military weapons or nuclear systems. For each dilemma, identify the most relevant principles from the ACM/IEEE Code of Ethics, evaluate the conflicting responsibilities, and propose a justified course of action that a professional software engineer should take.